

KASHÉ

Fitness Rewards Platform — System Design Document

Author	Esther Levy	Version	3.0
Phase 1	Class MVP — simulated attendance	Phase 2	Live launch — real studios & sponsors

1. Overview

Kashé is a cross-studio fitness loyalty platform. Customers earn points by attending classes at participating boutique studios and redeem those points for brand-sponsored rewards. Studios get a loyalty program at zero overhead; sponsors get high-intent product placement as the reward itself, not an ad interruption.

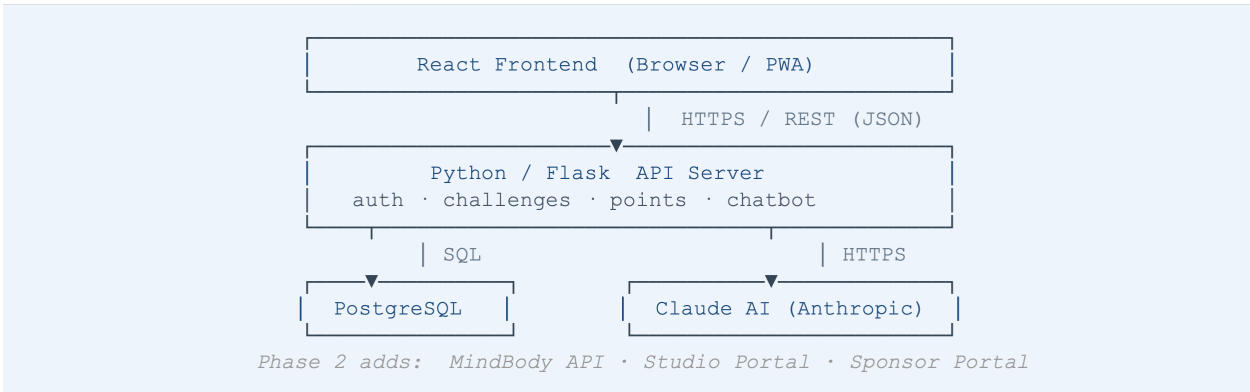
This document covers the full system in two phases. Phase 1 is the class MVP: a working web app with simulated attendance, core points logic, and an AI chatbot. Phase 2 extends it with real studio integrations and a sponsor portal. Phase 1 is architected so Phase 2 requires additions only — no rewrites.

2. Goals

Deliver three things: a loyalty experience that motivates customers, a frictionless path for studios to participate without changing their workflow, and a sponsor channel that earns Kashé revenue.

Feature	Phase 1 — MVP (Class)	Phase 2 — Launch
Auth	Register, log in, JWT session	No change needed
Challenges	Browse, join, track progress	Sponsor-linked, seasonal
Attendance	Manual "Log a Class" button (simulated)	Auto via MindBody API — same endpoint, new data source
Points	Earn on completion, spend on rewards	Same logic, more challenge types
Chatbot	Points balance + challenge progress	Expanded: reward recs, class suggestions
Portals	Not in scope	Studio admin + sponsor dashboard

3. High-Level Architecture



Component	Responsibility	Phase
React Frontend	Renders all screens. Makes fetch() calls to Flask. No business logic.	1 & 2
Flask API	All logic: auth, points, challenge lifecycle, chatbot orchestration.	1 & 2
PostgreSQL	Single source of truth for users, points, challenges, rewards.	1 & 2
Gemini API	LLM agent for chatbot. Flask exposes 12 tools (balance, enrollment, check-in, redemption, pace analysis, live weather via Open-Meteo) that Gemini calls autonomously.	1 & 2
MindBody API	Replaces manual check-in. Verified class attendance per user per studio.	2 only
Firebase Auth	verifies Google sign-in ID tokens server-side (Phase 1&2)	
AWS (EC2 + RDS + S3)	production hosting: API on EC2 under systemd/gunicorn, PostgreSQL on RDS, React build on S3 static hosting (Phase 1&2).	

Database Schema

```

users      (id, name, email, password_hash, mindbody_email)
challenges (id, title, required_classes, points_reward, deadline, sponsor_id, studio_id)
enrollments (id, user_id, challenge_id, classes_completed, status)
point_txns (id, user_id, delta, reason, created_at) ← balance = SUM(delta)
rewards    (id, title, points_cost, sponsor_id)
redemptions (id, user_id, reward_id, code, redeemed_at)

```

Database Seeding (Phase 1 MVP)

To support local development and testing, a seed.py script will populate the database with fake data on first setup. Running python seed.py will insert:

- 3–5 fake users with hashed passwords
- 4–6 challenges with varied required_classes, deadlines, and point rewards
- Pre-existing enrollments and partial progress for one test user
- A set of redeemable rewards linked to placeholder sponsors

The script uses SQLAlchemy and reads the same DATABASE_URL environment variable as the app, so it works identically in local and any hosted environment.

```

# Run once after flask db upgrade
python seed.py

```

4. Critical User Journeys (CUJs)

CUJ 1 — Register & Log In

User taps Register, fills in name / email / password, and lands on the Home screen. Returning users tap Log In.

```

POST /api/register { name, email, password }
  → hash pw, INSERT user, return { token }

POST /api/login { email, password }
  → verify hash, return { token }

```

All subsequent requests: Authorization: Bearer <token>

CUJ 2 — Browse & Join a Challenge

User taps Explore Challenges, sees a grid of active challenges, taps one, and taps Join. The challenge appears in My Challenges with a 0/N progress bar.

```
GET /api/challenges → SELECT active challenges, return list
POST /api/enroll { challenge_id } → INSERT enrollment (classes_completed=0)
```

CUJ 3 — Log a Class (Phase 1: button tap | Phase 2: automatic via MindBody)

In Phase 1 the user taps Log a Class on their challenge card. The endpoint is identical in Phase 2 — only the trigger changes from user tap to a nightly cron reading MindBody. No frontend rewrites required.

```
POST /api/checkin { enrollment_id }
  → increment classes_completed
  → if complete: UPDATE status='completed',
                  INSERT point_txns (delta=+points_reward)
  → return { classes_completed, completed, points_earned }
```

CUJ 4 — Claim a Reward

User opens Rewards tab, selects a reward they can afford, taps Claim, and receives a unique redemption code.

```
GET /api/rewards → rewards where points_cost <= user balance
POST /api/redeem { reward_id }
  → atomic transaction: INSERT redemption
                      + INSERT point_txns (delta=-points_cost)
  → return { code: 'KSH-4821', reward_title }
```

Note: balance always derived as `SELECT SUM(delta) FROM point_txns` — never stored.

CUJ 5 — AI Chatbot ♦ Designated Chatbot CUJ

Why this CUJ for the chatbot?

A rule-based bot could attempt this, but would require pre-writing every phrasing variant. Gemini handles natural language natively — and rather than injecting a snapshot of user data into a prompt, Flask exposes a set of 12 backend tools that Gemini calls autonomously, so every answer is computed from live database state and entire user journeys (enrolling, logging a class, redeeming a reward) can be completed in plain English through the chat.

User opens Chat and types "How many points do I have?", "Am I on pace to finish anything?", or even "Is it a good day to work out outside?" Flask passes the message to Gemini along with the tool catalog; Gemini decides which functions to call; Flask executes them against PostgreSQL (or the external Open-Meteo weather API) and the reply streams back to the UI over Server-Sent Events in ~1–2 seconds.

```
POST /api/chat/stream { message } (Auth: JWT)
```

```

Flask → Gemini: user message + 12-tool catalog
    (get_user_balance, get_user_challenges, enroll_in_challenge,
     log_class_for_challenge, redeem_reward_for_user,
     analyze_weekly_pace, suggest_weekly_plan,
     get_workout_weather ← external Open-Meteo API, ...)

Gemini → Flask: function call(s), e.g. get_user_balance(user_id)
Flask executes against PostgreSQL / external API → returns results
Gemini → natural-language reply, streamed via SSE

→ { reply: "You have 325 points! Two more classes for that
    free smoothie – and it's 75°F out, perfect for a run." }

```

5. UI Mockups

The following screens represent the core user-facing interface for Phase 1. Mockups to be added (Figma or hand-drawn) before final submission.

Home Screen

[MOCKUP PLACEHOLDER — Home Screen]

Shows the user's current point balance prominently, a summary of active challenge progress, and a shortcut to the Chat screen.

Explore Challenges Screen

[MOCKUP PLACEHOLDER — Explore Challenges Screen]

A grid or list of available challenges. Each card shows the challenge title, required classes, point reward, deadline, and a Join button. Enrolled challenges display a progress bar instead.

My Challenges / Log a Class Screen

[MOCKUP PLACEHOLDER — My Challenges Screen]

Lists the user's enrolled challenges. Each card has a Log a Class button (Phase 1) and shows classes_completed / required_classes progress. Completed challenges are grayed out with a checkmark.

Rewards Screen

[MOCKUP PLACEHOLDER — Rewards Screen]

Displays rewards the user can afford ($\text{points_cost} \leq \text{balance}$) highlighted, with unaffordable rewards dimmed. Tapping Claim triggers the redemption flow and shows the unique code.

Chat Screen

[MOCKUP PLACEHOLDER — Chat Screen]

A simple chat interface. User types a message at the bottom, send button on the right. Claude's replies appear as assistant bubbles. The screen opens with a greeting showing the user's current balance.

6. Decisions and Tradeoffs

Key decisions are summarized below using the following legend:

🚫 Blocker — cannot choose this | ! Serious issue | ⚠️ Mild issue | ✅ OK | 🟢 Recommended

Decision	Option A	Option B	Option C	Rationale
Attendance tracking	✅ Manual button (Phase 1)	✅ MindBody API (Phase 2)	🚫 Replace MindBody	Manual proves the logic fast with no third-party dependency. MindBody is the right Phase 2 path — studios already use it. Replacing it is premature. ⚠️ Known drawback (Phase 1): The manual button is unverified — a user can tap "Log a Class" without actually attending, accumulating points dishonestly. This is an accepted MVP limitation. Phase 2 MindBody integration eliminates the exploit by cross-referencing real studio check-in data.
Backend language	✅ Python / Flask	⚠️ Node.js		Developer knows Python. Flask has mature libraries for JWT, SQLAlchemy, and HTTP calls. Node.js is more common in React stacks but adds learning curve with no MVP benefit.
Points storage	✅ Transaction log (SUM)	! Balance column		A stored balance creates a double-write risk — crash between debit and update = corrupt data. An append-only transaction log is always consistent. Slow at scale, but a materialized view fixes that in Phase 2 without changing app logic.
Chatbot	✅ Claude + system prompt	⚠️ Rule-based bot		Claude handles any phrasing naturally. System prompt grounds it in real data, preventing hallucination. Rule-based requires maintaining patterns for every query variant and breaks on anything unexpected. 🕒 Updated June 2026: switched from Claude to Gemini with agentic tool-calling per professor's recommendation — instead of injecting data into a prompt, the model calls live backend functions, eliminating stale-data risk and enabling chat-driven actions.

Auth	✅ JWT (memory)	⚠️ JWT (localStorage)	⚠️ Session cookies	JWTs are stateless and simple. Storing in memory (not localStorage) blocks XSS token theft. Tokens can't be revoked early — add refresh token + revocation list before Phase 2 launch. ☹️ Added Google sign-in via Firebase Auth alongside JWT."
Webhook security	💜 Unauthenticated simulated webhook (Phase 1)	✅ HMAC-signed webhook (Phase 2)		the simulated MindBody webhook accepts unsigned requests, acceptable for the MVP simulation; production would verify an HMAC signature so attendance can't be forged.
Hosting	💜 EC2 + RDS + S3	⚠️ Single EC2 running everything		separate web/database machines mirror production architecture and satisfy independent scaling; S3 serves static files with no server to manage.

7. Milestones

#	Milestone	Deliverables	When
1	Setup & Auth	Flask project scaffold, PostgreSQL schema (all tables), /register and /login endpoints, JWT end-to-end. Run seed.py to populate fake users, challenges, and rewards for local testing.	Week 1
2	Core API — Challenges & Attendance	GET /api/challenges, POST /api/enroll, POST /api/checkin. Points logic: increment classes_completed, detect completion, write point_txns. Manually verify via Postman or curl.	Week 2
3	Core API — Rewards + React Frontend	GET /api/rewards, POST /api/redeem (atomic transaction). React screens: Home, Challenges, Rewards. CUJs 1–4 working end-to-end in the browser.	Week 3
4	AI Chatbot + Polish	Chat screen, POST /api/chat endpoint, Claude API integration with live user context. CUJ 5 working. Bug fixes, edge cases, final testing.	Week 4
5	Deployment	AWS EC2 + RDS + S3, systemd service, environment-based config	Week 5
6	Phase 2: Studios	MindBody integration. Nightly cron replaces manual check-in. Studio admin portal. Onboard 3 pilots.	Post-class
7	Phase 2: Sponsors	Sponsor portal, challenge + reward management, billing, production security hardening.	Post-class

This is a living document. When a decision changes — for example if a different booking platform replaces MindBody — update the Decisions section with the new reasoning rather than deleting the old entry. The history of why decisions changed is as valuable as the decisions themselves.